

HTB - Phantom Writeup

Author: Ethan Durnell

Date: 9/12/26

LinkedIn: <https://www.linkedin.com/in/ethandurnell/>

Summary & Intro

HackTheBox's (HTB) "sherlock" labs offer practice for multiple specialties of Blue Team Cybersecurity. The [Phantom lab](#) is a great exercise to begin dipping your toes into Digital Forensics and Incident Response (DFIR) and understanding some basic forensic processes. We'll analyze an infected server's memory dump using Volatility 3, a leading open-source DFIR tool and correlating information from multiple plugins of the tool. I will be including some of the errors and troubleshooting that I encountered during the lab so as to hopefully provide some fixes/inspiration for anyone running into the same issues.

My workstation is an ARM Macbook running a Kali docker image in parallel (when necessary), so some of my steps may be different if you're using Linux/Windows as your main driver.

Scenario & Evidence

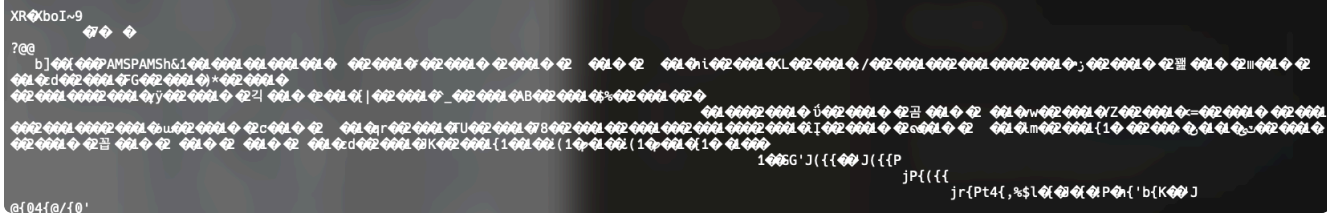
HTB provides the following scenario for this lab:

A Linux server in your organization has been exhibiting suspicious behavior. Network monitoring detected unusual outbound connections to an unknown IP address, and system administrators noticed that several standard diagnostic commands were returning incomplete information. A memory dump was captured from the compromised server before isolation. Your task is to analyze this memory dump to uncover evidence of a sophisticated rootkit infection, map its capabilities, and document all indicators of compromise.

The scenario comes with a `Phantom.zip` file attached. Unzipping and decrypting the file gives us two files. A 7 GB file titled `dump_srv.mem` and a small `Ubuntu_6.8.0-87-generic.json` file.

Unpacking the Samples

To try and understand what these files are, I wanted to grab a small sample of their text. Using the `cat` command to print the file's text, I piped (`|`) this into the `head` command to output a short sample. To output the first 20 lines of `dump_srv.mem` , I ran `cat dump_srv.mem | head -n 20` . This returned a blob of completely unintelligible text



This type of text blob is an indication that the file is likely a byte dump and is not (entirely) formatted for ASCII/UTF-8 display.

At this point, using context clues from the file name and scenario, it's clear that the `dump_srv.mem` file is a memory dump of the infected Linux server.

Since I'm new to DFIR, my first instinct was to run the `strings` command on the file with `strings dump_srv.mem` . After ~30 seconds of waiting for the function to complete, I realized that this was not going to be an effective method for analyzing a 7 GB file.

At this point, I turned my attention to the `Ubuntu_6.8.0-87-generic.json` file. Running `cat Ubuntu_6.8.0-87-generic.json | head -n 20` returned the following:

```
~/H/T/H/sherlock > cat Ubuntu_6.8.0-87-generic.json | head -n 20
{
  "metadata": {
    "linux": {
      "symbols": [
        {
          "kind": "dwarf",
          "name": "vmlinux-6.8.0-87-generic",
          "hash_type": "sha256",
          "hash_value": "d47721f218332487a200e83da970bcc6
b04e3208bd8bf31ea6a61ef3cc151f6a"
        },
        {
          "kind": "symtab",
          "name": "vmlinux-6.8.0-87-generic",
          "hash_type": "sha256",
          "hash_value": "d47721f218332487a200e83da970bcc6
b04e3208bd8bf31ea6a61ef3cc151f6a"
        }
      ],
      "types": [
        {
          "kind": "dwarf",
```

After a bit of googling, a [blog post](#) from user "Mihika" explained the use of symbol files very well.

*A **symbol file** in memory forensics is like a map or guide that helps tools like Volatility understand the structure of the operating system's memory. It contains information about functions, variables, and data structures used by the operating system. When analyzing a memory dump, the symbol file helps the tool accurately interpret the raw data, making it possible to identify processes, network connections, and other key details. Without the correct symbol file, the analysis might fail or give incorrect results.*

This blogpost also led me to look into the [Volatility](#) program for Digital Forensics. Volatility 3 is an open-source and widely used (4.4k stars on Github) Command Line Interface (CLI) tool for DFIR. Mihika's blog post mentions the use of Volatility as well, so I went ahead and proceeded to try and use this tool.

Setting up Volatility 3

Via the [Volatility Github](#), the program can be installed in a number of ways. After some toying around with installation methods, I found that downloading the Github repo to a local folder ultimately works best.

I found a [Volatility 3 Cheatsheet](#) published by Ashley Pearson to try and give me some beginner direction to start using the tool. The cheatsheet highlights that most of the tool is used via the `vol.py` file in the Github directory.

I attempted a test command `python3 vol.py -f "../dump_srv.mem windows.pslist` to see what results would be returned. In this command, `python3 vol.py` calls the `vol.py` script to run, the `-f` flag indicates which file is being analyzed, and the `windows.pslist` argument is the plugin we want to use.

This command returned the following output:

```
Volatility 3 Framework 2.28.0
Progress: 0.00      Scanning LimeLayer using PageMapScan
Progress: 0.00      Scanning LimeLayer using PageMapScan
Progress: 15.11     Scanning LimeLayer using PageMapScan
Progress: 0.00      Scanning LimeLayer using PageMapScan
Progress: 64.75     Scanning LimeLayer using PageMapScan
Progress: 0.00      Scanning LimeLayer using PageMapScan
Progress: 0.00      Scanning LimeLayer using PageMapScan
Progress: 0.00      Scanning LimeLayer using PageMapScan
Progress: 0.00      Scanning LimeLayer using PageMapScan
Progress: 2.69      Scanning LimeLayer using PageMapScan
Progress: 100.00    Stacking attempts finished
Progress: 100.00    PDB scanning finished

Unsatisfied requirement plugins.PsList.kernel.layer_name:
Unsatisfied requirement plugins.PsList.kernel.symbol_table_name:

A translation layer requirement was not fulfilled. Please verify th
at:
    A file was provided to create this layer (by -f, --single-lo
cation or by config)
    The file exists and is readable
    The file is a valid memory image and was acquired cleanly

A symbol table requirement was not fulfilled. Please verify that:
    The associated translation layer requirement was fulfilled
    You have the correct symbol file for the requirement
    The symbol file is under the correct directory or zip file
    The symbol file is named appropriately or contains the corre
ct banner

Unable to validate the plugin requirements: ['plugins.PsList.kernel.
layer_name', 'plugins.PsList.kernel.symbol_table_name']
```

The program output seemed to indicate that the symbol file (as mentioned earlier) was not loaded into the program properly. I attempted to use the `--symbol-dirs` flag to point to the symbol file with `python3 vol.py --symbol-dirs ../Ubuntu_6.8.0-87-generic.json -f ../dump_srv.mem windows.pslist`. This returned the same error.

After some digging in the [Volatility docs](#), I created a folder in the Volatility3 project file at `volatility3/symbols/mac` and put the file into there.

Once completed, I was able to receive some output from the following command, which verified that the tool should be working: `python3 vol.py -f ../dump_srv.mem banners.Banners`

Task 1

Task 1

What is the name of the hidden kernel module?

Submit Task

After doing some reading from TheXcellerator's [Github blog post](#) on Hiding Kernel Modules, it seems like hidden modules are present on the computer, but unrecognized by anything in the user space. For example, the author notes that the `lsmod` command should list all kernel modules, but a hidden module will evade detection.

From the [Volatility documentation](#), the tool is used by querying a number of pre-loaded plugins, each of which deriving different information from the memory dump. In order to get a full list of the plugins, the docs recommend running `python3 vol.py --help` for a list.

Since the [Scenario](#) states that this is a linux server, I wanted to find a list of linux-specific plugins with `python3 vol.py --help | grep -i linux`

```

    banners.Banners      Attempts to identify potential linux banner
s in an
    linux.bash.Bash      Recovers bash command history from memory.
    linux.boottime.Boottime
    linux.capabilities.Capabilities
    linux.check_afinfo.Check_afinfo
    linux.check_creds.Check_creds
    linux.check_idt.Check_idt
    linux.check_modules.Check_modules
    linux.check_syscall.Check_syscall
    linux.ebpf.EBPF      Enumerate eBPF programs
    linux.elfs.Elfs      Lists all memory mapped ELF files for all p
rocesses.
    linux.envvars.Envvars
    linux.graphics.fbdev.Fbdev
    linux.hidden_modules.Hidden_modules
    linux.iomem.IOMem    Generates an output similar to /proc/iomem
on a
    linux.ip.Addr        Lists network interface information for all
devices
    linux.ip.Link        Lists information about network interfaces
similar to

```

I funneled this into a text file for more convenience using `python3 vol.py --help | grep -i linux > ../vol3-linux.txt`, then ran `subl vol3-linux.txt` to browse. Note that I ran the same process for the full manual, in case we need any plugins that are not linux specific. This was done with `python3 vol.py --help > ../vol3.txt`, then `subl vol3.txt` to browse.

To search for a plugin that could assist in finding hidden kernel modules, I CTRL + F'd the document for the word "hidden", which highlighted the `linux.hidden_modules.Hidden_modules` plugin.

`python3 vol.py -f ../dump_srv.mem linux.malware.hidden_modules.Hidden_modules` returns one result.

```

0xfffffc0b42640 singularity 0x19000 OOT_MODULE,UNSIGNED_MODULE N/A

```

Task 1

What is the name of the hidden kernel module?

singularity



Submit Task

Task 2

Task 2

What kernel taint flags are set for the rootkit module? (comma-separated, alphabetical order)

Submit Task

The [Linux Kernel docs](#) detail:

A kernel will mark itself as 'tainted' when something occurs that might be relevant later when investigating problems"

Looking back at the results from the `linux.hidden_modules.Hidden_modules`, we see that there is a column titled "Taints", which lists two items: `OOT_MODULE` and `UNSIGNED_MODULE`.

```
Offset  Module Name      Code Size  Taints  Load Arguments  File Output
/Users/ethan/Home/Tech/HTB/sherlock/phantom/V3/volatility3/framework/deprecation.py:105: FutureWarning: This plugin (volatility3.plugins.linux.hidden_modules.Hidden_modules) has been renamed and will be removed in the first release after 2026-06-07. Please ensure all method calls to this plugin are replaced with calls to volatility3.plugins.linux.malware.hidden_modules.Hidden_modules
warnings.warn(
0xffffc0b42640  singularity      0x19000  OOT_MODULE,UNSIGNED_MODULE  N/A
```

Per the [Linux Kernel docs](#), `OOT_MODULE` likely details that this module is "Out of Tree", which indicates that it was externally created.

The `UNSIGNED_MODULE` tells us that the module was not cryptographically signed, which indicates that the author of the module did not have access to a valid signature key. Since we know that we're searching for a rootkit, this seems obvious, but in an initial investigation as part of Incident Response (IR), this would be strong evidence that connects an observed compromise with a rootkit.

Task 2

What kernel taint flags are set for the rootkit module? (comma-separated, alphabetical order)

`OOT_MODULE,UNSIGNED_MODULE` ✓

Submit Task

Task 3

Task 3

At what exact time (in seconds since boot) was the rootkit module loaded

Submit Task

The majority of this question (and many others in this lab) centered mostly around finding the right plugins to return the necessary information. I attempted anything that involved the word "time" or indication that the plugin would list modules: `linux.lsmmod.Lsmmod`, `timeliner.Timeliner`, `linux.malware.modxview.Modxview`.

Ultimately, I found that the `linux.kmsg.Kmsg` plugin prints the Kernel log buffer. When looking for meta-information surrounding actions on a computer, logs are always a good sign.

`python3 vol.py -f ../dump_srv.mem linux.kmsg.Kmsg` returned a significant amount of results.

CTRL + F for "singularity" showed the following logs:

```
kern warn 2490.473832 Task(2669) singularity: loading out-of-tree module taints kernel.  
kern notice 2490.473840 Task(2669) singularity: module verification failed: signature and/or required key missing - tainting kernel
```

At `2490.473832`, the "singularity: loading out-of-tree module taints kernel" entry indicates that the module attempted to be loaded.

Task 3

At what exact time (in seconds since boot) was the rootkit module loaded

2490.473832

Submit Task

Task 4

Task 4

What was the PID of the process that loaded the rootkit module?

number, such as 3, 17, or 4567

Submit Task

Frankly, the `Task(2669)` section from the previous `linux.kmsg.Kmsg` plugin logs looked too much like a PID for me to pass up trying.

```
kern warn 2490.473832 Task(2669) singularity: loading out-of-tree module taints kernel.  
kern notice 2490.473840 Task(2669) singularity: module verification failed: signature and/or required key missing - tainting kernel
```

Task 4

What was the PID of the process that loaded the rootkit module?

2669

Submit Task

Task 5

Task 5

Which kernel tracepoint is hooked by the rootkit

Submit Task

Once again, the [Linux Kernel docs](#) explain that tracepoints are essentially "hooks" to call functions on startup. This means that this particular rootkit "took" a tracepoint and will now run on startup. To the best of my understanding, this is an essential piece of rootkit architecture and is likely necessary to maintain persistence.

Looking through the plugin list, the `linux.tracing.tracepoints.CheckTracepoints` module seemed pretty self-explanatory.

`python3 vol.py -f ../dump_srv.mem linux.tracing.tracepoints.CheckTracepoints` returned the necessary information:

```
~/H/T/H/sh/p/V3 > python3 vol.py -f ../dump_srv.mem linux.tracing.tracepoints.CheckTracepoints
Volatility 3 Framework 2.28.0
Progress: 100.00 Stacking attempts finished
tracepoint      tracepoint address      Probe      Probe address      Probe priority Module      Module address
sched_process_fork 0xffff961fbae0 - 0xffffc0b3a990 10 singularity 0xffffc0b42640
```

From this, we see that the `sched_process_fork` tracepoint's "priority Module" is `singularity`.

Task 5

Which kernel tracepoint is hooked by the rootkit

`sched_process_fork` 

Submit Task

Task 6

Task 6

What is the IP address of the command and control server?

IPv4 address 

I first attempted to use the `linux.ip.Addr` and `linux.ip.Link` modules to list all network addresses and links, but the information was solely about local interfaces `lo` and `ens33` (a legacy `eth0` interface).

I attempted to run the `linux.sockscan.Sockscan` plugin to search for network connections in the memory layer, which found one external IP address `151{.}101{.}130{.}49`. I attempted to submit this, but it was incorrect.

The `linux.malware.netfilter.Netfilter` plugin didn't reveal anything meaningful.

Eventually, I found that the `linux.psaux.PsAux` plugin lists all processes, including their CLI arguments and some meta-information. CTRL + F for "singularity" showed that PID 2693, 2695, and 2698 were being used by `singularity`.

```

2693    1    bash    singularity
2695    1    bash    singularity
2696    2    kworker/2:3    [kworker/2:3]
2698    1    bash    singularity

```

I also found that the `linux.sockstat.Sockstat` shows which processes (and PIDs) are attached to network connections via sockets. CTRL + F for the PIDs shows that all of the `singularity` processes have an established connection to `192{.}168{.}200{.}164` over TCP.

```

4026531840    bash    2693    2693    0    0x8e7a08d2ef00    AF_INET    STREAM    TCP    192.168.200.177    48440    192.168.200.164    8081    ESTABLISHED    -
4026531840    bash    2693    2693    1    0x8e7a08d2ef00    AF_INET    STREAM    TCP    192.168.200.177    48440    192.168.200.164    8081    ESTABLISHED    -
4026531840    bash    2693    2693    2    0x8e7a08d2ef00    AF_INET    STREAM    TCP    192.168.200.177    48440    192.168.200.164    8081    ESTABLISHED    -
4026531840    bash    2695    2695    0    0x8e7a08d2dc80    AF_INET    STREAM    TCP    192.168.200.177    48444    192.168.200.164    8081    ESTABLISHED    -
4026531840    bash    2695    2695    1    0x8e7a08d2dc80    AF_INET    STREAM    TCP    192.168.200.177    48444    192.168.200.164    8081    ESTABLISHED    -
4026531840    bash    2695    2695    2    0x8e7a08d2dc80    AF_INET    STREAM    TCP    192.168.200.177    48444    192.168.200.164    8081    ESTABLISHED    -
4026531840    bash    2698    2698    0    0x8e7a08d2ae40    AF_INET    STREAM    TCP    192.168.200.177    48454    192.168.200.164    8081    ESTABLISHED    -
4026531840    bash    2698    2698    1    0x8e7a08d2ae40    AF_INET    STREAM    TCP    192.168.200.177    48454    192.168.200.164    8081    ESTABLISHED    -
4026531840    bash    2698    2698    2    0x8e7a08d2ae40    AF_INET    STREAM    TCP    192.168.200.177    48454    192.168.200.164    8081    ESTABLISHED    -

```

Task 6

What is the IP address of the command and control server?

192.168.200.164



Submit Task

Task 7

Task 7

What port is the C2 server listening on?

number, such as 3, 17, or 4567

Submit Task

From the results of the `linux.sockstat.Sockstat` plugin, we see that the C2 server is listening on port 8081. Per whatportis.com, 8081 is a nonstandard, alternate HTTP port used for management consoles and web services. This information is likely irrelevant for our investigation, but was likely chosen as a form of stealth.

4026531840	bash	2693	2693	0	0x8e7a08d2ef00	AF_INET	STREAM	TCP	192.168.200.177	48440	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2693	2693	1	0x8e7a08d2ef00	AF_INET	STREAM	TCP	192.168.200.177	48440	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2693	2693	2	0x8e7a08d2ef00	AF_INET	STREAM	TCP	192.168.200.177	48440	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2695	2695	0	0x8e7a08d2dc80	AF_INET	STREAM	TCP	192.168.200.177	48444	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2695	2695	1	0x8e7a08d2dc80	AF_INET	STREAM	TCP	192.168.200.177	48444	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2695	2695	2	0x8e7a08d2dc80	AF_INET	STREAM	TCP	192.168.200.177	48444	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2698	2698	0	0x8e7a08d2ae40	AF_INET	STREAM	TCP	192.168.200.177	48454	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2698	2698	1	0x8e7a08d2ae40	AF_INET	STREAM	TCP	192.168.200.177	48454	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2698	2698	2	0x8e7a08d2ae40	AF_INET	STREAM	TCP	192.168.200.177	48454	192.168.200.164	8081	ESTABLISHED	-

Task 7

What port is the C2 server listening on?

8081

✓

Submit Task

Task 8

Task 8

What are the PIDs of the compromised bash processes connected to the C2 server? (comma-separated, ascending order)

Submit Task

Again, these PIDs were listed in the `linux.sockstat.Sockstat` plugin results:

4026531840	bash	2693	2693	0	0x8e7a08d2ef00	AF_INET	STREAM	TCP	192.168.200.177	48440	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2693	2693	1	0x8e7a08d2ef00	AF_INET	STREAM	TCP	192.168.200.177	48440	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2693	2693	2	0x8e7a08d2ef00	AF_INET	STREAM	TCP	192.168.200.177	48440	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2695	2695	0	0x8e7a08d2dc80	AF_INET	STREAM	TCP	192.168.200.177	48444	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2695	2695	1	0x8e7a08d2dc80	AF_INET	STREAM	TCP	192.168.200.177	48444	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2695	2695	2	0x8e7a08d2dc80	AF_INET	STREAM	TCP	192.168.200.177	48444	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2698	2698	0	0x8e7a08d2ae40	AF_INET	STREAM	TCP	192.168.200.177	48454	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2698	2698	1	0x8e7a08d2ae40	AF_INET	STREAM	TCP	192.168.200.177	48454	192.168.200.164	8081	ESTABLISHED	-
4026531840	bash	2698	2698	2	0x8e7a08d2ae40	AF_INET	STREAM	TCP	192.168.200.177	48454	192.168.200.164	8081	ESTABLISHED	-

Task 8

What are the PIDs of the compromised bash processes connected to the C2 server? (comma-separated, ascending order)

2693,2695,2698

✓

Submit Task

Task 9

Task 9

How many hooks has the rootkit installed?

number, such as 3, 17, or 4567

Submit Task

I looked through two modules that I thought would contain this information, but neither of them did.

`linux.malware.check_syscall.Check_syscall` . [Syscalls](#) are the method for computers to request a service from the OS. Since the rootkit is located in the OS, this module could have had information on how many services were called from the rootkit. It didn't.

`linux.malware.tty_check.Tty_Check` . [TTYs in Linux](#) are text-only terminals that are used as a way to get access to a computer. Frankly, I only used this module as the description indicated some form of "hooking". This would not have any information that we're looked for since tracepoint hooks don't involve text-only terminals.

Lastly, I found the `linux.tracing.ftrace.CheckFtrace` module, as the documentation referenced "hooks" in it. [Ftrace](#) "is an internal tracer designed to help out developers and designers of systems to find what is going on inside the kernel".

`python3 vol.py ../dump_srv.mem linux.tracing.ftrace.CheckFtrace` returned a significant amount of information. Each piece of information is a process, the relevant memory addresses, and what it is hooked to. Looking at the results, there are a significant amount of entries for `singularity` .

```
0xfffffc0b40000 - 0xfffffc0b3aac0 __x64_sys_newfstatat singularity 0xfffffc0b35000
0xfffffc0b3ff20 - 0xfffffc0b3aac0 __ia32_sys_getpriority singularity 0xfffffc0b35000
0xfffffc0b3fe40 - 0xfffffc0b3aac0 __x64_sys_getpriority singularity 0xfffffc0b35000
0xfffffc0b3fd60 - 0xfffffc0b3aac0 __ia32_sys_newlstat singularity 0xfffffc0b35000
0xfffffc0b3fc80 - 0xfffffc0b3aac0 __x64_sys_newlstat singularity 0xfffffc0b35000
0xfffffc0b3fba0 - 0xfffffc0b3aac0 __ia32_sys_newstat singularity 0xfffffc0b35000
0xfffffc0b3fac0 - 0xfffffc0b3aac0 __x64_sys_newstat singularity 0xfffffc0b35000
0xfffffc0b3f9e0 - 0xfffffc0b3aac0 __ia32_sys_lstat singularity 0xfffffc0b35000
0xfffffc0b3f900 - 0xfffffc0b3aac0 __x64_sys_lstat singularity 0xfffffc0b35000
0xfffffc0b3f820 - 0xfffffc0b3aac0 __ia32_sys_stat singularity 0xfffffc0b35000
0xfffffc0b3f740 - 0xfffffc0b3aac0 __x64_sys_stat singularity 0xfffffc0b35000
0xfffffc0b3f660 - 0xfffffc0b3aac0 __ia32_sys_statx singularity 0xfffffc0b35000
0xfffffc0b3f580 - 0xfffffc0b3aac0 __x64_sys_statx singularity 0xfffffc0b35000
0xfffffc0b3e300 - 0xfffffc0b3aac0 __ia32_compat_sys_getdents singularity 0xfffffc0b35000
```

While I could have worked some CLI magic to count these, CTRL + F for "singularity" showed that 82 hooks were connected.

Task 9

How many hooks has the rootkit installed?

82



Submit Task

Task 10

Task 10

Which function is hooked to hide IPv4 network connections?

**** _ _ ****

Submit Task

When manually looking through the `linux.tracing.ftrace.CheckFtrace` plugin connections, a `tcp4_seq_show` function is hooked to `singularity` and directly references TCP IPv4. Especially when contrasted with the `tcp6_seq_show`, which references TCP IPv6, it seems like this is the culprit.

```
__x86_sys_ftrace_module singularity 0xffffffffc0b35000
tpacket_rcv singularity 0xffffffffc0b35000
tcp6_seq_show singularity 0xffffffffc0b35000
tcp4_seq_show singularity 0xffffffffc0b35000
```

After looking through the other hooks, no others directly reference IPv4.

Task 10

Which function is hooked to hide IPv4 network connections?

tcp4_seq_show



Submit Task

Task 11

Task 11

How many variants of getdents syscalls are hooked?

number, such as 3, 17, or 4567

Submit Task

By using the `linux.tracing.ftrace.CheckFtrace` plugins results from earlier and CTRL + F for "getdents", five results are returned:

```
0xfffffc0b3e300 - 0xfffffc0b3aac0 __ia32_compat_sys_getdents singularity 0xfffffc0b35000
0xfffffc0b3e220 - 0xfffffc0b3aac0 __ia32_sys_getdents singularity 0xfffffc0b35000
0xfffffc0b3e140 - 0xfffffc0b3aac0 __ia32_sys_getdents64 singularity 0xfffffc0b35000
0xfffffc0b3e060 - 0xfffffc0b3aac0 __x64_sys_getdents singularity 0xfffffc0b35000
0xfffffc0b3df80 - 0xfffffc0b3aac0 __x64_sys_getdents64 singularity 0xfffffc0b35000
```

Via the [Linux man pages](#), the `getdents` syscall is used by user-space applications to read the contents of a directory. CLI functions like `ls` directly call this function. With this information, it's likely that directory exploration/navigation is used at some point in the rootkit's payload.

Task 11

How many variants of getdents syscalls are hooked?

5



Submit Task

Task 12

Task 12

Which function is hooked to enable an ICMP-based covert channel?

**** **
_

Submit Task

When looking at the `linux.tracing.ftrace.CheckFtrace` plugin results again, the `icmp_rcv` hook is the only one that directly mentions `icmp` :

ftrace_ops	address	Callback	Callback address	Hooked symbols	Module	Module address
0xfffffc0b422c0	-	0xfffffc0b3aac0	icmp_rcv	singularity	0xfffffc0b35000	

A [blog post](#) from Private Island Networks notes that `icmp_rcv` is the kernel process that controls and manages allowing ICMP traffic to the computer. It's possible that the rootkit used this to force open a ICMP channel for Command and Control (C2) traffic or data exfiltration.

Task 12

Which function is hooked to enable an ICMP-based covert channel?

`icmp_rcv`



Submit Task

Task 13

Task 13

What is the memory address of the centralized callback function? (Format:0x*****)

Submit Task

Again, when searching through the `linux.tracing.ftrace.CheckFtrace` plugin results and CTRL + F for "call", `call_direct_funcs` was the only option that showed up.

0x8e7a03dec180	call_direct_funcs	0xffff93a8aa20	__hid_bpf_tail_call	__kernel__	0xffff93800000
----------------	-------------------	----------------	---------------------	------------	----------------

I was confident that the `0x8e7a03dec180` address for the function would be correct, but it was not. Turns out, the FTrace plugin has a whole column for Callback addresses, and they all call back to `0xfffffc0b3aac0`.

ftrace_ops	address	Callback	Callback address	Hooked symbols	Module	Module address
0xfffffc0b422c0	-	0xfffffc0b3aac0	icmp_rcv	singularity	0xfffffc0b35000	
0xfffffc0b42160	-	0xfffffc0b3aac0	__ia32_sys_bpf	singularity	0xfffffc0b35000	
0xfffffc0b42080	-	0xfffffc0b3aac0	__x64_sys_bpf	singularity	0xfffffc0b35000	
0xfffffc0b41b80	-	0xfffffc0b3aac0	__ia32_sys_readlink	singularity	0xfffffc0b35000	
0xfffffc0b41aa0	-	0xfffffc0b3aac0	__x64_sys_readlink	singularity	0xfffffc0b35000	
0xfffffc0b419c0	-	0xfffffc0b3aac0	__ia32_sys_chdir	singularity	0xfffffc0b35000	

Task 13

What is the memory address of the centralized callback function? (Format:0x*****)

0xffffc0b3aac0



Submit Task

Task 14

Task 14

What is the value of the suspicious environment variable which leads to the escalation of privileges?

Submit Task

According to the Volatility docs, the `linux.envvars.Envvars` plugin lists processes with their environment variables.

`python3 vol.py -f ../dump_srv.mem linux.envvars.Envvars` returns a large list of all processes and their environment variables.

Looking for the the PIDs used by the rootkit (2693, 2695, 2698), we start to see some arbitrary variables.

```
2693 1 bash PWD /
2693 1 bash HOME /
2693 1 bash TERM xterm-256color
2693 1 bash SHLVL 0
2693 1 bash PATH /usr/bin:/bin:/usr/sbin:/sbin
2695 1 bash PWD /
2695 1 bash HOME /
2695 1 bash TERM xterm-256color
2695 1 bash SHLVL 0
2695 1 bash PATH /usr/bin:/bin:/usr/sbin:/sbin
2698 1 bash PWD /
2698 1 bash HOME /
2698 1 bash TERM xterm-256color
2698 1 bash SHLVL 0
```

None of the variables from our PIDs seemed to relate to privilege escalation, so I began to look around at other PIDs.

PID 2774 shows that an SSH session was established with the C2 server address from before:

```
2774 2773 bash SSH_CLIENT 192.168.200.1 63600 22
2774 2773 bash SSH_CONNECTION 192.168.200.1 63600 192.168.200.177 22
2774 2773 bash SSH_TTY /dev/pts/0
2792 2774 bash OPERATOR access
```

Looking through PID 2774's environment variables further, between the SSH connection and the launch of the shell in `SHELL /bin/bash`, the `access` variable is mentioned.

```
2792 2774 bash OPERATOR access
2792 2774 bash SHELL /bin/bash
2792 2774 bash PWD /home/workstation
2792 2774 bash LOGNAME workstation
2792 2774 bash XDG_SESSION_TYPE tty
2792 2774 bash HOME /home/workstation
2792 2774 bash LANG pt_PT.UTF-8
```

It's safe to assume that this was some file, script, or process that elevated privileges.

Task 14

What is the value of the suspicious environment variable which leads to the escalation of privileges?

access



Submit Task